



L.J. INSTITUTES OF COMPUTER APPLICATIONS									
Programme	Master of Computer Applications					Branch / Spec.	Computer Applications		
Semester	II					Effective from Academic Year		2025-26	
Subject Category	Major					Effective for the batch Admitted in		June 2025	
Subject Code	240110201		Subject Name		Data Structure (DS)				
Teaching scheme					Examination scheme (Marks)				
(Lecture hours Per week)	Credit	L	T	P	Total	E	CEC	V	Total
3	3	3	0	0	36	50	50	0	100
University Exam Evaluation Scheme									
Internal Evaluation					External Evaluation				
Mid Sem Theory Exam(30 Marks)		30			Theory Exam		50		
Attendance		10			Practical Exam		NA		
Theory Assignment		10							
Total Marks		50			Total Marks		50		
Pre-requisites:									
- Basic knowledge of writing and understanding algorithms for solving a problem. - Knowledge of programming in C language									
Program Learning Outcome:									
Name of PO		Description							
PO1		Foundational & Theoretical Mastery							
PO2		Software Design & Development Expertise							
PO3		Problem-Solving & Analytical Ability							
PO4		Proficiency in Emerging Technologies							
PO5		Information Systems & Management Perspective							
PO6		Professional & Ethical Responsibility							
PO7		Communication & Teamwork							
PO8		Lifelong Learning & Career Readiness							
Course Learning Outcome:									
Name of CO		Description							
CO1		To understand how to solve a problem step by step to get the desired output with different way.							



CO2	To learn the optimal way to organize data in the digital space.
CO3	To understand the fundamental concept of elementary data structures and their implementation.
CO4	To understand where and how the data structures are implemented to solve the real world problems.
CO5	To learn how to write efficient and optimized computer programs.

Mapping of CO and PO:

Course Learning Objectives	Program Outcomes (POs)							
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8
CO1	3	2	2	-	-	-	-	1
CO2	2	3	3	-	-	-	-	1
CO3	2	3	3	-	2	2	-	1
CO4	2	3	3	2	-	2	-	1
CO5	1	3	2	2	-	-	2	1

Content:

Unit	Content	Weightage (%)	No of Hrs.	CO
Unit I	Introduction to Data Structures Data types – primitive and non-primitive, abstract data type, concept of data structures, what is algorithm?, introduction to linear and nonlinear data structures, introduction to calculation of time complexity (best case, worst case and average case) and space complexity.	15%	6	CO1
Unit II	Linear Data Structures: Array, Linked lists Array: Operations (row major and column major address calculations) Linked List: Basic concept, difference between linked list and array, storage representation, types of linked lists (singly, doubly, circular), operations on linked list (insert, modify, delete, union, intersection, merge, sort, searching, reverse (by changing address not data)), applications of linked list (polynomial operations – addition), sparse matrices (only concept).	20%	8	CO2



Unit III	Linear Data Structures: Stacks, Queues Stack: Basic concept, storage representation (array and linked list), basic operations (push, pop, peep and change), applications of stacks (polish and reverse polish expressions), arithmetic expression evaluation using stack. Queues: Basic concept, storage representation (array and linked list), basic operations (insert and delete). Types of queues- circular, deque, priority queues (only concept). Application of queues.	20%	8	CO3
Unit IV	Non Linear Data Structures – Trees and Graphs. Trees – Basic concept, terms associated with trees (Node, parent, child, link, root, leaf, level, height, indegree, outdegree, siblings), Storage representation – Linear and Linked, Conversion of General tree to Binary tree, Complete Binary tree, full binary tree, BST operations, Tree traversals – In-order, Pre-order, Post-order, Types of Binary tree : BST (Practical – recursive), Expression tree, AVL Tree (without practical), 2-3 tree, B tree (without practical)) Graphs -Basic concepts, technical terms associated with Graphs – Digraph, Weighted graph, adjacent vertices, self loop, parallel edges, simple graph, complete graph, isolated vertex, Degree of a vertex, connected graph-, Storage representation (Set representation, Adjacency matrix, Adjacency list), Graph Traversing algorithms- DFS and BFS. Spanning Tree, Minimum spanning tree algorithm: Prim's Algorithm, Kruskal's Algorithm, Shortest path algorithm – Dijkstra's Algorithm (without practical)	30%	8	CO4
Unit V	Sorting and Searching Algorithms and Hashing Sorting Algorithms: Bubble, Selection, Insertion, Shell, 2- Way Merge sort, Radix sort, heap sort and Quick sort. Searching Algorithms: Binary Search, Hashing: Introduction to Hashing and hashing methods, collision resolution techniques.	15%	6	CO5



Reference Books:

1	Debasis Samanta, Classic Data Structures , PHI, Second Edition.
2	Jean-Paul Tremblay, Paul G. Sorenson, "An Introduction to Data Structures with Applications", Tata McGraw-Hill, 2nd Edition, (2007).
3	Mark Allen Weiss, “Data Structures and Algorithm Analysis in C”, Pearson, Second Edition,
4	Ashok N. Kamthane, “Introduction to Data Structures in C”, Pearson Education (2004).
5	Reema Thareja, Data Structures using C, Oxford

Web Reference:

1	https://www.tutorialspoint.com/data_structures_algorithms
2	https://www.javatpoint.com/
3	GeeksforGeeks (Data Structures & Algorithms): https://www.geeksforgeeks.org

Question Paper Scheme:

	University Examination Duration: 2 Hours 30 minutes. Q-1: 10 Marks Q-2: 10 Marks Q-3: 10 Marks Q-4: 10 Marks Q-5: 10 Marks
--	--



L.J. INSTITUTES OF COMPUTER APPLICATIONS									
Programme		Master of Computer Applications				Branch/Spec.		Computer Applications	
Semester		II				Effective from Academic Year		2025-26	
Subject Category		Major				Effective for the batch Admitted in		June 2025	
Subject Code		240110202		Subject Name		Data Structure Lab (DS-Lab)			
Teaching scheme						Examination scheme (Marks)			
(Lecture hours Per week)	Credit	L	T	P	Total	E	CEC	V	Total
4	2	0	0	4	48	0	50	50	100
University Exam Evaluation Scheme									
Internal Evaluation						External Evaluation			
Mid Sem (50 Marks)			30			Theory Exam		NA	
Attendance			10			Practical Exam		50	
Hacker Rank Practical Assignment			10						
Total Marks			50			Total Marks		50	
Pre-requisites:									
- Basic knowledge of writing and understanding algorithms for solving a problem. - Knowledge of programming in C language.									
Program Learning Outcome:									
Name of PO		Description							
PO1		Foundational & Theoretical Mastery							
PO2		Software Design & Development Expertise							
PO3		Problem-Solving & Analytical Ability							
PO4		Proficiency in Emerging Technologies							
PO5		Information Systems & Management Perspective							
PO6		Professional & Ethical Responsibility							
PO7		Communication & Teamwork							
PO8		Lifelong Learning & Career Readiness							
Course Learning Outcome:									
Name of CO		Description							
CO1		To understand how to solve a problem step by step to get the desired output with different way.							



CO2	To learn the optimal way to organize data in the digital space.
CO3	To understand the fundamental concept of elementary data structures and their implementation.
CO4	To understand where and how the data structures are implemented to solve the real world problems.
CO5	To learn how to write efficient and optimized computer programs.

Mapping of CO and PO:

Course Learning Objectives	Program Outcomes (POs)							
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8
CO1	3	2	2	-	-	-	-	1
CO2	2	3	3	-	-	-	-	1
CO3	2	3	3	-	2	2	-	1
CO4	2	3	3	2	-	2	-	1
CO5	1	3	2	2	-	-	2	1

Content:

Unit	Content	Weightage (%)	No of Hrs.	CO
Unit I	1. Write a program to reverse a number. 2. Write a program to find sum of digits 3. Write a program to display array elements 4. Write a program to find sum and average of array elements 5. Write a program to find largest and smallest element in array 6. Write a program to access array elements using pointers. 7. Write a program to allocate memory using malloc() 8. Write a program to allocate memory using calloc()	10%	6	CO1
Unit II	1. Write a program to find the total of each rows and columns in a given matrix. 2. Program to reverse an array and store it in another array. 3. Write a menu driven program to perform following operations for an array a) Print b) Insert at a specific position c) Delete at a specific position d) Update at specific position e) Exit 4. Write a program for linear search in array 5. Write a program using singly linked list to perform following operations: a) Create b) Display c) Insert - First, Last, At a specified location d) Delete -First, Last, At a specified location 6. Perform the following operations on singly linked list using user-defined functions: a) Count total nodes b) Create a copy of the linked list c) Merge 7. Perform the following operations on singly linked list using	25%	12	CO2



	user-defined functions: a) Reverse a listed list b) Search for a specific node c) Sort 8. Write a program to count the number of occurrences of a user specified element in the singly linked list. If that element does not present give an appropriate message 9. Write a program to display all the nodes of a singly linked list which is divisible by 7. If not found, then display appropriate message. 10. Write a program to display all the even node of a singly linked list. If not found, then display appropriate message 11. Write a program to search an existing element in a singly linked list. If not found, then display appropriate message 12. Implement Stack using linked list 13. Implement simple queue using linked list 14. Write a program using doubly linked list to perform following operations a) Create b) Display c) Insert (At all locations) d) Delete (At all locations) e) Count f) Exit 15. Write a program using doubly linked list to perform following operations a) Copy b) Search 16. Write a program using doubly linked list to perform merge operation 17. 14. Write a program using circular linked list to perform following operations a) Create b) Display c) Insert (At all locations) d) Delete (At all locations) e) Count f) Exit			
Unit III	1. Write a menu driven program to perform following operations in stack using static data structure. Also display the updated value of Top in every operation. a) Push b) Pop c) Peep d) Count e) Display f) Exit 2. Write a program to reverse a given number using stack. 3. Write a program to convert infix expression to postfix 4. Write a program to evaluate arithmetic expression from postfix expression. 5. Write a program to check balanced parenthesis in expression using stack 6. Write a menu driven program to perform following operations in simple queue using static data structure. Also display the updated value of Front and Rear in every operation. a) Insert b) Delete c) Display d) Exit 7. Write a menu driven program to perform following operations in circular queue using static data structure. Also display the updated value of Front and Rear in every operation. a) Insert b) Delete c) Display d) Exit	25%	12	CO3
Unit IV	1. Write program to create and display binary tree 2. Write program to create and display binary search tree 3. Write a program to display tree traversals	20%	10	CO4



	In-order traversal (recursive) Pre-order traversal (recursive) Post-order traversal (recursive) 4. Write a program to create a graph using adjacency matrix 5. Write a program to display degree of each vertex in a graph			
Unit V	1. Write a program for following sorting techniques. a) Bubble b) Selection c) Insertion d) Shell e) Merge f) Quick 2. Write a program implement binary search algorithm	20%	8	CO5

Reference Books:

1	Debasis Samanta, Classic Data Structures , PHI, Second Edition.
2	Jean-Paul Tremblay, Paul G. Sorenson, "An Introduction to Data Structures with Applications", Tata McGraw-Hill, 2nd Edition, (2007).
3	Mark Allen Weiss, "Data Structures and Algorithm Analysis in C", Pearson, Second Edition,
4	Ashok N. Kamthane, "Introduction to Data Structures in C", Pearson Education (2004).
5	Reema Thareja, Data Structures using C, Oxford

Web Reference:

1	https://www.tutorialspoint.com/data_structures_algorithms
2	https://www.javatpoint.com/
3	GeeksforGeeks (Data Structures & Algorithms): https://www.geeksforgeeks.org

Question Paper Scheme:

	University Examination Duration: 2 Hours 30 minutes. SECTION-I Q-1: (Unit I & II) (15 Marks) Q-2: (Units III & IV)(15 Marks) Q-3: (Unit V). (10 Marks) SECTION-II VIVA (10 Marks)
--	--



L.J. INSTITUTES OF COMPUTER APPLICATIONS									
Programme	Master of Computer Applications					Branch/Spec.	Computer Applications		
Semester	II					Effective from Academic Year	2025-26		
Subject Category	Major					Effective for the batch Admitted in	June 2025		
Subject Code	240110203		Subject Name		Object Oriented Programming with JAVA (OOPJ)				
Teaching scheme					Examination scheme (Marks)				
(Lecture hours Per week)	Credit	L	T	P	Total	E	CEC	V	Total
6	4	0	1	6	72	0	50	50	100
University Exam Evaluation Scheme									
Internal Evaluation					External Evaluation				
Mid Sem (50 Marks)		20			Theory Exam		NA		
Mid Sem MCQ Exam (30 Marks)		20							
Attendance		10			Practical Exam		50		
Total Marks		50			Total Marks		50		
Pre-requisites:									
Knowledge of the C programming language.									
Program Learning Outcome:									
Name of PO	Description								
PO1	Foundational & Theoretical Mastery								
PO2	Software Design & Development Expertise								
PO3	Problem-Solving & Analytical Ability								
PO4	Proficiency in Emerging Technologies								
PO5	Information Systems & Management Perspective								
PO6	Professional & Ethical Responsibility								
PO7	Communication & Teamwork								
PO8	Lifelong Learning & Career Readiness								
Course Learning Outcome:									
Name of CO	Description								
CO1	Explain and apply object-oriented programming concepts and Java fundamentals to develop basic Java programs using classes, objects, methods, and constructors.								



	application development.
CO2	Implement inheritance and polymorphism concepts in Java using abstract classes, method overriding, and dynamic binding to design reusable and extensible applications.
CO3	Develop Java applications using interfaces, exception handling mechanisms, and collection framework classes to build robust and modular programs..
CO4	Design and implement multithreaded Java applications and perform file handling operations using Java I/O streams with proper synchronization.
CO5	Create interactive GUI-based applications using Java Swing components, layout managers, and event-handling mechanisms.

Mapping of CO and PO:

Course Learning Objectives	Program Outcomes (POs)							
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8
CO1	3	2	2	-	-	-	-	1
CO2	2	3	3	-	-	-	-	1
CO3	2	3	3	-	2	2	-	1
CO4	2	3	3	2	-	2	-	1
CO5	1	3	2	2	-	-	2	1

Content:

Unit	Content	Weightage (%)	No of Hrs.	CO
Unit I	Basics of OOPL & Class: Basics: Object-Oriented Programming concepts (object, class, encapsulation, abstraction, inheritance, polymorphism, message passing, and dynamic binding), features of Java, Java's magic: byte code. (First day lab activity: Understanding Java Development Kit (JDK) and its settings, compiling and executing the first simple program, comments, Java class libraries, arrays, difference between C & Java) Class: Class fundamentals, the general form of a class, defining a class, creating objects, working with method, constructors (default, parameterized, copy constructor), garbage collector, this keyword, static block, initializer block A Closer Look at Methods and Classes: Passing object to the method, returning object from the method, method overloading, constructor overloading, static variable, static method, nested and inner classes, local class, anonymous class	20%	15	CO1
Unit II	Inheritance & Polymorphism: Inheritance & Polymorphism: Inheritance, constructors and	20%	14	CO2



	inheritance, types of inheritance, uses of super keyword for constructor and method, creating a multilevel hierarchy, constructor's execution sequence, superclass references and subclass objects, method overriding, polymorphism, abstract class, final keyword, Object class and its methods.			
Unit III	Interfaces, Exception & Collection: Interfaces: Interface, working with interface, multiple inheritance with interface Exception: Exception hierarchy, exception handling fundamentals, the consequences of an uncaught exception, using multiple catch statements, catching subclass exception, nested try blocks, re-throwing an exception, finally & throws keywords, java's built-in exceptions, user define exception Collection Framework : Collection, Collection framework, Collection interface, Iterator interface, list interface, stack interface, vector interface	20%	12	CO3
Unit IV	Multi-Threading, Generic, I/O Multithreaded Programming: Multithreading fundamentals, the thread class and runnable interface, creating a thread, creating multiple threads, determining thread methods, thread priorities, need for synchronization, thread synchronization Using I/O: Working with File class, creating file & folder, Renaming file & folder, Deleting file & folder, byte streams and character streams, the byte stream classes, the character stream classes, the predefined streams, using the byte streams (reading console input, writing console output), reading and writing files using byte streams (inputting from a file, writing to a file), automatically closing a file, reading and writing binary data, random access file	20%	15	CO4
Unit V	GUI Programming with Swing: Introduction: GUI Programming, AWT Components, swing components, comparing AWT and swing, event handling mechanism. Swing components with event handling: JFrame, JPanel, JButton, Layout Managers (FlowLayout, BorderLayout, GridLayout, GridBagLayout, CardLayout), JLabel, JTextField, JTextArea, JPasswordField, JCheckBox, JRadioButton, JComboBox, JList, JScrollBar, Adapter classes, dialog boxes, working with menus	20%	16	CO5
Reference Books:				
1	Schildt, Herbert and Coward, Danny. Java: The Complete Reference, 13th Edition, McGraw Hill Education, 2024.			



2	Schildt, Herbert and Coward, Danny. Java: A Beginner's Guide, 10th Edition, McGraw Hill Education
3	Horstmann, Cay S. Core Java, Volume I – Fundamentals, Latest Edition, Pearson Education.
4	Pandey, Hari Mohan. <i>Java Programming</i> . Pearson Education (India), 2011
5	Bloch, Joshua. Effective Java, 3rd Edition, Addison-Wesley Professional.

Web Reference:

1	Oracle Java Documentation – https://docs.oracle.com/en/java/
2	GeeksforGeeks – Java Programming -- https://www.geeksforgeeks.org/java/
3	Baeldung – Java Tutorials https://www.baeldung.com/java-tutorial

Question Paper Scheme:

University Examination Duration: 2 Hours 30 minutes. SECTION-I Q-1: Classes, Methods, Objects (Unit I & II) (10 Marks) - Classes, Objects, Constructors (default, parameterized, copy) - Method overloading - Passing and returning objects - Inheritance and polymorphism (if applicable) Q-2: File I/O and Collections (Units III & IV) (10 Marks) - Accept data from the user (e.g., student marks, employee details) - Store and retrieve data using File I/O - Implement exception handling for invalid inputs - Use at least one Java Collection (List, Stack, Vector) for data storage Q-3: GUI Programming with Swing (Unit V). (20 Marks) Implement a full Swing GUI program with: - JFrame, JPanel, JLabel, JTextField, JTextArea, JButton, JCheckBox, JRadioButton, JComboBox - Event handling for buttons or interactive components - Layout Managers (FlowLayout, BorderLayout, GridLayout) - Integrate file I/O or simple multithreading SECTION-II VIVA (10 Marks)	
---	--

SNo	Practical List
Unit-I	Basics of OOPL & Class:
1	Hello World – Print Message Definition: Write a Java program to display the message “Hello World” on the console. This program introduces the basic structure of a Java program, including class declaration,



	the main method, and using <code>System.out.println()</code> to print output. It also demonstrates how to compile and execute a Java program to generate the class file.
2	Concepts: if-else, conditional logic, input/output (take input from command line) 1. Check Number is Odd or Even (if-else). 2. Write a program in Java to find maximum of three numbers using conditional operator. 3. Write a program in Java to print following pattern using for loop. <pre>* ** *** ****</pre>
3	Concepts: for loop, while loop, iteration, arithmetic operations(take input from Scanner class) 1. Write a java program to check Armstrong number (take input using Scanner class) a. Input: 153 Output: Armstrong number b. Input: 22 Output: not Armstrong number 2. Write a Java program to calculate and display the sum of first N natural numbers using for loop. 3. Write a Java program to reverse a given number using a while loop.
4	Concepts: switch case, menu-driven program, control statements 1. Write a Java program to perform basic arithmetic operations (Addition, Subtraction, Multiplication, Division) using a menu-driven approach with switch case. Concepts: Array, Class and Object (Basic) 1. Write a Java program to calculate and display the sum and average of elements stored in a one-dimensional array. 2. Write a Java program to store values in a two-dimensional array and display them in matrix form. 3. Write a Java program to display student details such as name, age, and hobby on the console.
5	Concepts: working with methods. 1. Write a Java program to create a class named <code>Car</code> to represent a real-world car. Include the following requirements: • Define data members: <code>brand</code>, <code>model</code>, and <code>year</code> • Create a method <code>displayDetails()</code> to print the car details • In the <code>main</code> method, create two objects of the <code>Car</code> class with different data • Call the <code>displayDetails()</code> method for both objects to display their information



2. Write a Java program that defines a class named `Calculator`. Create the following methods in the class:

- `add(int a, int b)` – returns the sum of two numbers
- `subtract(int a, int b)` – returns the difference of two numbers
- `multiply(int a, int b)` – returns the product of two numbers

In the `main` method, create an object of the `Calculator` class, call all the methods, and display their results.

Concepts: method overloading

Write a Java program to demonstrate **method overloading** by creating a class named `MathOperations`.

Define the following overloaded methods named `calculate`:

- `calculate(int a, int b)` – returns the sum of two integers
- `calculate(double a, double b)` – returns the sum of two decimal numbers
- `calculate(int a, int b, int c)` – returns the sum of three integers

In the `main` method, create an object of the `MathOperations` class, call all the overloaded `calculate` methods with different arguments, and display the results.

6 **Concepts:** constructor (default, parameterized , copy constructor)and constructor overloading

1. Write a Java program to demonstrate **constructor overloading** by creating a class named `Student`.

- Define the following overloaded constructors in the class:
- A constructor that accepts **no arguments** and initializes default values for name and age
- A constructor that accepts **one argument** (name)
- A constructor that accepts **two arguments** (name and age)
- Create objects of the `Student` class using each constructor and display the student details.

2. Write a Java program to demonstrate the use of a **constructor** by creating a class named `Employee`.

Include the following requirements:



	• Define data members: <code>employeeName</code> and <code>employeeId</code> • Create a constructor to initialize the data members • Create a method <code>displayEmployee()</code> to display employee details • In the <code>main</code> method, create two objects of the <code>Employee</code> class using the constructor • Call the <code>displayEmployee()</code> method for both objects to display their information
7	Concepts: static variable and static method 1. Write a Java program to demonstrate the use of a static variable and a static method by creating a class named <code>College</code>. Include the following requirements: • Declare a static variable <code>collegeName</code> that is shared by all objects of the class • Declare non-static variables such as <code>studentName</code> and <code>studentId</code> • Create a constructor to initialize the non-static variables • Create a static method <code>displayCollegeName()</code> to display the college name • Create a non-static method <code>displayStudentDetails()</code> to display student details • In the <code>main</code> method, call the static method using the class name and create multiple objects to display student details
8	Concepts: nested class , inner class ,local class and anonymous class 1. Write a Java program to demonstrate the use of a nested class by creating a class named <code>Company</code>. Include the following requirements: • Create an outer class <code>Company</code> with a variable <code>companyName</code> • Create a static nested class named <code>Department</code> inside the <code>Company</code> class • The nested class should contain a variable <code>departmentName</code> and a method to display department details • In the <code>main</code> method, create an object of the static nested class using the outer class name • Display the company name and department details 2. Write a Java program to demonstrate the use of a non-static inner class by creating a class named <code>Library</code>. Include the following requirements: • Create an outer class <code>Library</code> with a variable <code>libraryName</code>



	• Create a non-static inner class named <code>Book</code> inside the <code>Library</code> class • The inner class should have variables <code>bookTitle</code> and <code>authorName</code> and a method to display book details • In the <code>main</code> method: • Create an object of the outer class • Use it to create an object of the inner class • Display the library name and book details 3. Write a Java program to demonstrate the use of a local class by creating a class named <code>School</code>. Include the following requirements: • Create an outer class <code>School</code> with a method <code>displayStudents()</code> • Inside the <code>displayStudents()</code> method, define a local class named <code>Student</code> • The local class should have variables <code>studentName</code> and <code>studentGrade</code> and a method to display student details • Create an object of the local class inside the method and display the student details • In the <code>main</code> method, create an object of the <code>School</code> class and call the <code>displayStudents()</code> method 4. Write a Java program to demonstrate the use of an anonymous class by creating an interface named <code>Greeting</code>. Include the following requirements: • Create an interface <code>Greeting</code> with a method <code>sayHello()</code> • In the <code>main</code> method, implement the interface using an anonymous class • Override the <code>sayHello()</code> method to print a greeting message • Call the <code>sayHello()</code> method using the anonymous class object
9	Concepts: this keyword(with variable, constructor,) and toString() method 1. Write a Java program to demonstrate the use of the this keyword. Perform the following tasks: • Create a class named DemoThis.



	• Declare an instance variable <code>int value</code>. • Create a parameterized constructor <code>DemoThis(int value)</code> and use the <code>this</code> keyword to assign the parameter value to the instance variable. • Create a method <code>display()</code> and use the <code>this</code> keyword to call another method of the same class. • Create a method <code>show(DemoThis obj)</code> and pass the current object using the <code>this</code> keyword. • Create a method <code>getObject()</code> that returns the current object using the <code>this</code> keyword. • In the <code>main()</code> method, create an object of the class and demonstrate all the above uses of the <code>this</code> keyword. 2. Write a Java program to demonstrate the use of the <code>this</code> keyword. • Perform the following tasks: • Create a class named <code>DemoThis</code>. • Declare an instance variable <code>int value</code>. • Create a parameterized constructor <code>DemoThis(int value)</code> and use the <code>this</code> keyword to assign the parameter value to the instance variable. • Create a method <code>display()</code> and use the <code>this</code> keyword to call another method of the same class. • Create a method <code>show(DemoThis obj)</code> and pass the current object using the <code>this</code> keyword. • Create a method <code>getObject()</code> that returns the current object using the <code>this</code> keyword. • In the <code>main()</code> method, create an object of the class and demonstrate all the above uses of the <code>this</code> keyword.
10	Concepts: single level and multilevel inheritance 1. Write a Java program to demonstrate single-level inheritance by creating two classes: <code>Vehicle</code> and <code>Car</code>. Include the following requirements: • Create a parent class <code>Vehicle</code> with: 1. Data members: <code>brand</code> and <code>year</code> 2. Method: <code>displayVehicle()</code> to display vehicle details • Create a child class <code>Car</code> that inherits from <code>Vehicle</code> and adds: 1. Data member: <code>model</code> 2. Method: <code>displayCar()</code> to display car details including the inherited vehicle details



	- In the <code>main</code> method, create an object of the <code>Car</code> class and call both methods to display details. - Write a Java program to demonstrate multi-level inheritance by creating three classes: <code>Vehicle</code>, <code>Car</code>, and <code>ElectricCar</code>. Include the following requirements: - Create a base class <code>Vehicle</code> with: - Data members: <code>brand</code> and <code>year</code> - Method: <code>displayVehicle()</code> to display vehicle details - Create a derived class <code>Car</code> that inherits from <code>Vehicle</code> and adds: - Data member: <code>model</code> - Method: <code>displayCar()</code> to display car details - Create another derived class <code>ElectricCar</code> that inherits from <code>Car</code> and adds: - Data member: <code>batteryCapacity</code> - Method: <code>displayElectricCar()</code> to display electric car details - In the <code>main</code> method, create an object of <code>ElectricCar</code> and call all the display methods to show details from all classes.
11	Concepts: hierarchical inheritance - Write a Java program to demonstrate hierarchical inheritance by creating a base class <code>Vehicle</code> and two derived classes <code>Car</code> and <code>Bike</code>. Include the following requirements: - Create a base class <code>Vehicle</code> with: - Data members: <code>brand</code> and <code>year</code> - Method: <code>displayVehicle()</code> to display vehicle details - Create a derived class <code>Car</code> that inherits from <code>Vehicle</code> and adds: - Data member: <code>model</code> - Method: <code>displayCar()</code> to display car details - Create another derived class <code>Bike</code> that inherits from <code>Vehicle</code> and adds: - Data member: <code>type</code> (e.g., <code>Sports</code>, <code>Cruiser</code>) - Method: <code>displayBike()</code> to display bike details - In the <code>main</code> method, create objects of both <code>Car</code> and <code>Bike</code> classes and call the respective methods to display details. Concepts: <code>super</code> keyword for method.



	2. Write a Java program to demonstrate the use of the super keyword for calling a method of the parent class. Include the following requirements: • Create a parent class <code>Vehicle</code> with a method <code>display()</code> that prints "This is a vehicle" • Create a child class <code>Car</code> that extends <code>Vehicle</code> and overrides the <code>display()</code> method to print "This is a car" • Inside the <code>display()</code> method of <code>Car</code>, use the super keyword to call the parent class <code>display()</code> method before printing the child class message • In the <code>main</code> method, create an object of <code>Car</code> and call the <code>display()</code> method
12	Concepts: super keyword for constructor. 1. Write a Java program to demonstrate the use of the super keyword to call a parent class constructor. Include the following requirements: • Create a parent class <code>Vehicle</code> with: 1. A constructor that accepts a parameter <code>brand</code> and prints "Vehicle Brand: " followed by the brand name • Create a child class <code>Car</code> that extends <code>Vehicle</code> with: 1. A constructor that accepts parameters <code>brand</code> and <code>model</code> 2. Use the super keyword to call the parent class constructor for <code>brand</code> 3. Print "Car Model: " followed by the model name • In the <code>main</code> method, create an object of the <code>Car</code> class and pass the brand and model values Concepts: constructor execution sequence. 2. Write a Java program to demonstrate the execution sequence of constructors in single-level and multi-level inheritance. Include the following requirements: 1. Create a parent class <code>Vehicle</code> with a constructor that prints "Vehicle Constructor Called"



	2. Create a child class <code>Car</code> that extends <code>Vehicle</code> with a constructor that prints "Car Constructor Called" 3. Create another child class <code>ElectricCar</code> that extends <code>Car</code> with a constructor that prints "ElectricCar Constructor Called" 4. In the <code>main</code> method, create an object of the <code>ElectricCar</code> class and observe the sequence of constructor execution
13	Concepts: superclass references and subclass objects 1. Write a Java program to demonstrate using a superclass reference to refer to a subclass object. Include the following requirements: • Create a superclass <code>Vehicle</code> with: 1. Method <code>start()</code> that prints "Vehicle is starting" • Create a subclass <code>Car</code> that extends <code>Vehicle</code> with: 1. Method <code>start()</code> that overrides the parent method and prints "Car is starting" 2. Method <code>displayCar()</code> that prints "This is a car" • In the <code>main</code> method: 1. Declare a superclass reference of type <code>Vehicle</code> 2. Assign it an object of the <code>Car</code> class 3. Call the <code>start()</code> method using the superclass reference 4. Try to call <code>displayCar()</code> using the superclass reference and observe the behavior Concepts: method overriding 2. Write a Java program to demonstrate method overriding by creating a superclass <code>Vehicle</code> and a subclass <code>Car</code>. Include the following requirements: • Create a superclass <code>Vehicle</code> with: 1. Method <code>fuelType()</code> that prints "Vehicles can use different types of fuel" • Create a subclass <code>Car</code> that extends <code>Vehicle</code> with: 1. Method <code>fuelType()</code> that overrides the parent method and prints "Cars use petrol or diesel" • In the <code>main</code> method: 1. Create an object of the <code>Vehicle</code> class and call <code>fuelType()</code> 2. Create an object of the <code>Car</code> class and call <code>fuelType()</code>



	3. Observe the output to understand method overriding
14	Concepts: final keyword 1. Write a Java program to demonstrate the use of the final keyword in Java. Include the following requirements: • Final Variable: 1. Create a class <code>BankAccount</code> with a final variable <code>bankName</code> initialized to "ABC Bank" 2. Try to change the value of <code>bankName</code> and observe what happens • Final Method: 1. Create a method <code>displayBank()</code> in <code>BankAccount</code> and declare it as final 2. Create a subclass <code>SavingsAccount</code> that attempts to override <code>displayBank()</code> and observe the behavior • Final Class: 1. Create a final class <code>FixedDeposit</code> with a method <code>showDetails()</code> 2. Try to extend <code>FixedDeposit</code> in another class and observe the behavior Concepts: abstract class 2. Write a Java program to demonstrate the use of an abstract class by creating a class named <code>Shape</code>. Include the following requirements: • Create an abstract class <code>Shape</code> with: 1. An abstract method <code>area()</code> to calculate the area of the shape 2. A non-abstract method <code>display()</code> that prints "This is a shape" • Create two subclasses <code>Circle</code> and <code>Rectangle</code> that extend <code>Shape</code>: 1. <code>Circle</code> should have a data member <code>radius</code> and implement the <code>area()</code> method 2. <code>Rectangle</code> should have data members <code>length</code> and <code>width</code> and implement the <code>area()</code> method • In the <code>main</code> method: 1. Create objects of <code>Circle</code> and <code>Rectangle</code> 2. Call the <code>area()</code> method to display the area of each shape 3. Call the <code>display()</code> method from the abstract class
15	Concepts: polymorphism



Write a Java program to demonstrate **polymorphism** (both **compile-time** and **runtime**) using classes `Animal`, `Dog`, and `Cat`.

Include the following requirements:

1. Compile-Time Polymorphism (Method Overloading):

- Create a class `Calculator` with a method `add(int a, int b)`
- Overload the method to `add(int a, int b, int c)`
- In the `main` method, call both methods and display the results

2. Runtime Polymorphism (Method Overriding):

- Create a superclass `Animal` with a method `sound()` that prints "Animal makes a sound"
- Create subclasses `Dog` and `Cat` that override the `sound()` method to print "Dog barks" and "Cat meows" respectively
- In the `main` method, create a **superclass reference** for `Dog` and `Cat` objects and call the `sound()` method

Concepts: object class and its methods

Write a Java program to demonstrate the use of the **object class** and some of its important methods.

Include the following requirements:

1. Create a class `Person` with data members `name` and `age`.
2. Override the `toString()` method to return person details in a readable format.
3. Override the `equals(Object obj)` method to compare two `Person` objects based on `name` and `age`.
4. In the `main` method:
 - Create two `Person` objects with the same or different data
 - Use the `toString()` method to print their details
 - Use the `equals()` method to check if the two objects are equal
 - Use the `hashCode()` method to print the hash codes of both objects

16

Concepts: interface

Write a Java program to demonstrate the use of an **interface** by creating an interface named `Drawable`.

Include the following requirements:

1. Create an **interface** `Drawable` with a method `draw()`
2. Create two classes `Circle` and `Rectangle` that **implement** the `Drawable` interface
 - `Circle` should implement `draw()` to print "Drawing a Circle"



- Rectangle should implement `draw()` to print "Drawing a Rectangle"
- 3. In the `main` method:
 - Create objects of `Circle` and `Rectangle`
 - Call the `draw()` method on both objects

Concepts: multiple inheritance using interface

Write a Java program to demonstrate **multiple inheritance** using **interfaces** by creating interfaces `Vehicle` and `Electric`.

Include the following requirements:

1. Create an **interface** `Vehicle` with a method `start()` that prints "Vehicle is starting"
2. Create another **interface** `Electric` with a method `charge()` that prints "Vehicle is charging"
3. Create a class `ElectricCar` that **implements both interfaces** `Vehicle` and `Electric`
 - Implement the `start()` and `charge()` methods in `ElectricCar`
4. In the `main` method:
 - Create an object of `ElectricCar`
 - Call both `start()` and `charge()` methods

17

Concepts: exception handling

Write a Java program to demonstrate the **exception hierarchy** in Java by handling different types of exceptions.

Include the following requirements:

1. Create a `try` block to perform the following operations:
 - Divide an integer by zero (to cause `ArithmeticException`)
 - Access an invalid index in an array (to cause `ArrayIndexOutOfBoundsException`)
2. Create multiple `catch` blocks:
 - One for `ArithmeticException`
 - One for `ArrayIndexOutOfBoundsException`
 - One for the parent class `Exception` to catch any other exceptions
3. Include a `finally` block to print "Execution completed"
4. In the `main` method, run the program and observe the order in which exceptions are caught

Concepts: uncaught exception



	Write a Java program to demonstrate the consequences of an uncaught exception. Include the following requirements: 1. Create a <code>main</code> method that performs the following operations without using a <code>try-catch</code> block: ○ Divide an integer by zero (causes <code>ArithmeticException</code>) ○ Access an invalid index in an array (causes <code>ArrayIndexOutOfBoundsException</code>) 2. Run the program and observe what happens when the exceptions occur. 3. Add comments explaining: ○ How the program terminates abruptly ○ Which line caused the exception ○ The stack trace printed by the JVM
18	Concepts: multiple catch statements Write a Java program to demonstrate the use of multiple catch statements. Include the following requirements: 1. Create a <code>try</code> block to perform the following operations: ○ Divide an integer by zero (causes <code>ArithmeticException</code>) ○ Access an invalid index in an array (causes <code>ArrayIndexOutOfBoundsException</code>) ○ Parse a non-numeric string to an integer (causes <code>NumberFormatException</code>) 2. Create separate <code>catch</code> blocks for: ○ <code>ArithmeticException</code> ○ <code>ArrayIndexOutOfBoundsException</code> ○ <code>NumberFormatException</code> 3. In each <code>catch</code> block, print an appropriate message indicating which exception occurred. 4. Include a <code>finally</code> block to print "Program execution completed" Concepts: catching subclass exception Write a Java program to demonstrate catching a subclass exception in Java. Include the following requirements: 1. Create a <code>try</code> block that performs the following operations:



	○ Access an invalid index in an array (causes <code>ArrayIndexOutOfBoundsException</code>) ○ Divide an integer by zero (causes <code>ArithmeticException</code>) 2. Create multiple <code>catch</code> blocks: ○ First, catch the subclass exception <code>ArrayIndexOutOfBoundsException</code> ○ Then, catch the parent exception <code>Exception</code> 3. In each <code>catch</code> block, print a message indicating which exception was caught. 4. Include a <code>finally</code> block that prints "Execution finished"
19	Concepts: nested try blocks Write a Java program using nested try–catch blocks that performs the following tasks: 1. Declare two integers and perform division. 2. Inside the outer <code>try</code> block, use an inner try block to access an array element. 3. Handle the following exceptions: ○ <code>ArithmeticException</code> in the outer catch block ○ <code>ArrayIndexOutOfBoundsException</code> in the inner catch block 4. Display appropriate messages for each exception. 5. Ensure the program continues execution after exception handling. Concepts: re-throwing an exception Write a Java program that demonstrates re-throwing an exception with the following requirements: 1. Create a method that performs division of two numbers. 2. Handle <code>ArithmeticException</code> inside the method. 3. After displaying an error message, re-throw the same exception to the calling method. 4. Catch the re-thrown exception in the <code>main()</code> method. 5. Display appropriate messages at each level of exception handling
20	Concepts: throws keyword Write a Java program that demonstrates the use of the throws keyword by performing the following tasks: 1. Create a method that accepts a student's age. 2. If the age is less than 18, throw an <code>ArithmeticException</code>. 3. Declare the exception using the throws keyword in the method definition.



	4. Handle the exception in the main() method. 5. Display appropriate messages for valid and invalid age. Concepts: java's built in exception Write a Java program to demonstrate the handling of Java's built-in exceptions by performing the following tasks: 1. Perform division of two numbers to demonstrate <code>ArithmeticException</code>. 2. Access an array element using an invalid index to demonstrate <code>ArrayIndexOutOfBoundsException</code>. 3. Convert a String to an integer to demonstrate <code>NumberFormatException</code>. 4. Handle each exception using separate catch blocks. 5. Display meaningful messages for each exception and allow the program to continue execution
21	Concepts: user defined exception Write a Java program to demonstrate a user-defined exception by performing the following tasks: 1. Create a custom exception class named <code>InvalidAgeException</code>. 2. The exception should be thrown when a person's age is less than 18. 3. Create a method that checks the age and throws the user-defined exception. 4. Handle the exception in the <code>main()</code> method using a <code>try-catch</code> block. 5. Display appropriate messages for valid and invalid age. 6. Ensure normal program termination Concepts: collection Write a Java program to demonstrate the use of Java Collections by performing the following tasks: 1. Create an <code>ArrayList</code> to store student names. 2. Add at least five student names to the list. 3. Display all elements using an Iterator. 4. Remove one student name from the list. 5. Check whether a particular student name exists in the list.
22	Concepts: collection framework



	Write a Java program to demonstrate the use of the Collection Framework by performing the following operations: 1. Create an <code>ArrayList</code> to store integer values. 2. Add at least five elements to the list. 3. Display all elements using an Iterator. 4. Remove a specific element from the list. 5. Search for an element using the <code>contains()</code> method. 6. Sort the elements in ascending order. 7. Display the final list after all operations Concepts: collection interface Write a Java program to demonstrate the use of the Collection interface by performing the following operations: 1. Create a <code>Collection</code> object using an implementing class (<code>ArrayList</code>). 2. Add at least five elements to the collection. 3. Display all elements using an Iterator. 4. Remove a specific element from the collection. 5. Check whether a given element exists using the <code>contains()</code> method. 6. Display the size of the collection. 7. Display the final elements of the collection.
23	Concepts: interface iterator Write a Java program to demonstrate the use of the Iterator interface by performing the following operations: 1. Create a collection of integer elements using <code>ArrayList</code>. 2. Add at least five elements to the collection. 3. Traverse and display all elements using an Iterator. 4. Remove a specific element from the collection using the Iterator. 5. Display the elements after removal. 6. Ensure safe traversal without causing <code>ConcurrentModificationException</code> Concepts: list interface Concepts: list interface



	Write a Java program to demonstrate the use of the List interface by performing the following operations: 1. Create a List of String elements using an implementing class (ArrayList). 2. Add at least five elements to the list. 3. Display all elements using: - o a for-each loop - o an Iterator 4. Insert an element at a specific index. 5. Remove an element using its index value. 6. Retrieve and display an element using the get () method. 7. Display the final list and its size
24	Concepts: stack interface Write a Java program to demonstrate the use of Stack in Java by performing the following operations: 1. Create a Stack of Integer elements. 2. Push at least five elements into the stack. 3. Display the stack elements. 4. Perform pop operation. 5. Perform peek operation. 6. Search for an element in the stack. 7. Display the final contents of the stack. Concepts: vector interface Write a Java program to demonstrate the use of Vector by performing the following operations: 1. Create a Vector of String elements. 2. Add at least five elements to the vector. 3. Display all elements using an Enumeration. 4. Insert an element at a specific index. 5. Remove an element from the vector. 6. Display the size and capacity of the vector. 7. Display the final contents of the vector.
25	Concepts: Thread class Write a Java program to demonstrate the use of the Thread class by performing the following tasks:



	1. Create a class that extends <code>Thread</code>. 2. Override the <code>run()</code> method to display a message five times with a short delay. 3. Create two thread objects and start both threads. 4. Observe how threads execute concurrently. Concepts: Runnable interface Write a Java program to demonstrate the use of the Runnable interface by performing the following tasks: 1. Create a class that implements <code>Runnable</code>. 2. Override the <code>run()</code> method to display a message five times with a short delay. 3. Create two thread objects using <code>Thread</code> class and start both threads. 4. Observe how threads execute concurrently.
26	Concepts: creating and running multiple threads Write a Java program to demonstrate creating and running multiple threads in Java by performing the following tasks: 1. Create three threads by extending the <code>Thread</code> class. 2. Override the <code>run()</code> method for each thread to display a message five times with a short delay. 3. Start all threads and observe concurrent execution. 4. Display a message from the main thread after starting all threads Concepts: thread methods Write a Java program to demonstrate the use of commonly used thread methods in Java, including: 1. <code>getName()</code> – to get the thread name. 2. <code>setName()</code> – to set a thread name. 3. <code>getId()</code> – to get the thread ID. 4. <code>getPriority()</code> – to get thread priority. 5. <code>setPriority()</code> – to set thread priority. 6. <code>isAlive()</code> – to check if a thread is alive. 7. <code>sleep()</code> – to pause thread execution. Create at least two threads and display their details before and after starting them
27	Concepts: thread priorities



Write a Java program to demonstrate the use of **thread priorities** in Java by performing the following tasks:

1. Create **three threads** by extending the `Thread` class.
2. Override the `run()` method to display a message **five times** with a short delay.
3. Assign different priorities to each thread using `setPriority()`.
4. Display thread names and their priorities.
5. Start all threads and observe how priorities affect execution.

Concepts: thread synchronization

Write a Java program to demonstrate **thread synchronization** in Java by performing the following tasks:

1. Create a class `BankAccount` with a shared resource `balance`.
2. Create a `deposit` method that adds money to the balance.
3. Create a `withdraw` method that deducts money from the balance.
4. Use `synchronized` keyword to ensure **thread-safe operations**.
5. Create **two threads** performing deposit and withdrawal simultaneously.
6. Display the final balance after all transactions.



L.J. INSTITUTES OF COMPUTER APPLICATIONS									
Programme	Master of Computer Applications					Branch/Spec.	Computer Applications		
Semester	II					Effective from Academic Year		2025-26	
Subject Category	Major					Effective for the batch Admitted in		June 2025	
Subject Code	240110204		Subject Name		Python Programming (Python)				
Teaching scheme						Examination scheme (Marks)			
(Lab hours Per week)	Credit	L	T	P	Total	E	CEC	V	Total
6	3	0	0	6	72	0	50	50	100
University Exam Evaluation Scheme									
Internal Evaluation						External Evaluation			
Mid Sem (50 Marks)		30				Theory Exam		NA	
Attendance		10				Practical Exam		50	
Hacker Rank Practical Assignment		10							
Total Marks		50				Total Marks		50	
Pre-requisites:									
Knowledge of the C programming language.									
Program Learning Outcome:									
Name of PO		Description							
PO1		Foundational & Theoretical Mastery							
PO2		Software Design & Development Expertise							
PO3		Problem-Solving & Analytical Ability							
PO4		Proficiency in Emerging Technologies							
PO5		Information Systems & Management Perspective							
PO6		Professional & Ethical Responsibility							
PO7		Communication & Teamwork							
PO8		Lifelong Learning & Career Readiness							
Course Learning Outcome:									
Name of CO		Description							
CO1		To understand the basics of Python, including its history, applications, advantages, and disadvantages. Set up the Python environment and work with both interactive and							



	script modes, including Python tokens, keywords, and identifiers.
CO2	To implement conditional statements, loops, and functions in Python. Learn about recursion and exception handling techniques, including the use of try, except, raise, and finally blocks to ensure robust code.
CO3	To understand and work with Python's core data structures (lists, tuples, dictionaries, and strings). Perform file operations such as reading from and writing to text files, and handle CSV and Excel files using Python
CO4	To use Pandas to manipulate and analyze data, including handling missing data, grouping, merging, and aggregating. Create visualizations using Matplotlib to represent data through various plots (scatter, bar charts, histograms, etc.).
CO5	To integrate Python concepts learned (file handling, data analysis, visualization, and database connectivity) to develop real-world data-driven applications. This includes using MySQL for database interaction and creating practical solutions to problems.

Mapping of CO and PO:

Course Learning Objectives	Program Outcomes (POs)							
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8
CO1	3	2	0	3	0	0	2	2
CO2	3	3	0	3	0	0	0	0
CO3	3	2	0	3	0	0	0	0
CO4	0	2	0	3	2	0	0	0
CO5	2	3	0	3	0	0	0	3

Content:

Unit	Content	Weightage (%)	No of Hrs.	CO
Unit I	Introduction to Python and Core Basics - History and need of Python - Applications of Python - Advantages and Disadvantages of Python - Installing Python - Program Structure - Interactive Shell - Executable / Script files - User Interface / IDE (IDLE, SPYDER) - Working with Interactive Mode - Working with Script Mode Python Fundamentals - Python Character Set - Python - Tokens - Keywords and Identifiers - Variables , Constants and Assignments	20%	14	CO1



	Basic Data Types - Numbers Representing binary, octal and hexadecimal numbers in python - Strings - Basic operations on data types - Introduction to Mutable and Immutable Data Types Basic Input and Output in Python Operators and Expressions Basic Operators –Arithmetic operators, Relational operators, Logical operators, Membership operators, Ternary operators, Boolean operators, Bitwise Operators. - Type Conversion - Expressions and Evaluation - Formatted Input and Output			
Unit II	Program Control Flow and Python Data Structures Conditional Statements - if - if-else - if-elif - Nested if - Python Indentation Looping and Iteration - range() function - Introduction - Types - Use of range() - for loop - while loop - loop-else statement - Nested loops - Break and Continue Immutable Data Structures – Strings, Tuples Mutable Data structures – Sets, Lists and Dictionaries	20%	14	CO2
Unit III	Functions and Files in Python Functions - Built-in Functions - Introduction to Functions - Structure of Python Functions - User Defined Functions (UDF) - Arguments and Parameters - Default Arguments - Named Arguments - Scope of Variables - Lambda Functions	20%	14	CO3



	• Recursion Functions File Operations • Text and Byte Files • Opening Files • Reading and Writing Files • Other File Tools • Working with MS Excel Files ◦ .xls ◦ .csv			
Unit IV	Exception Handling and Database Connectivity and GUI Exception Handling • Default Exceptions and Errors • try-except statement • raise statement • assert • finally block • User Defined Exceptions Database Connectivity • Introduction to MySQL • PyMySQL Connections • Executing Queries • Transactions • Handling Database Errors	20%	14	CO4



UNIT V	Data Science using Python- Pandas and Matplotlib Pandas • Importing data from ◦ CSV ◦ Excel ◦ HDF5 ◦ SQL Databases • Series and DataFrames • Multilevel Series • Grouping and Aggregation • Generating Summary Tables • Data Wrangling • Merging and Joining Data Frames • Date and Time Manipulation • Creating Metrics for Analysis Data Visualization using Matplotlib • Scatter Plot • Bar Chart • Histogram • Stack Charts • Legends, Titles, Styles • Figures and Subplots • Plotting with Pandas • Labelling and Arranging Figures • Saving Plots	20%	14	CO5
---------------	---	------------	-----------	------------



Reference Books:	
1	Core python programming by Dr. R. Nageshwar Rao
2	Pandas for Everyone by Daniel Y. Chen
3	McKinney Wes, "Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython", O'Reilly Media, 2012.
4	Hauck Trent, "Instant Data Intensive Apps with Pandas How-To", Packt Publishing Ltd, 2013.
Web Reference:	
1	https://www.python.org
2	https://www.w3schools.com/python
Question Paper Scheme:	
	University Examination Duration: 2 Hours 30 minutes. SECTION-I Q-1: From Unit-1 and Unit-2 Q.2: From Unit-3 Q.3: From Unit-4 SECTION-II VIVA (10 Marks)



L.J. INSTITUTES OF COMPUTER APPLICATIONS									
Programme	Master of Computer Applications					Branch/Spec.	Computer Applications		
Semester	II					Effective from Academic Year		2025-26	
Subject Category	Minor					Effective for the batch Admitted in		June 2025	
Subject Code	240111205		Subject Name		Software Engineering and Testing (SE)				
Teaching scheme						Examination scheme (Marks)			
(Lecture hours Per week)	Credit	L	T	P	Total	E	CEC	V	Total
4	4	3	1	0	48	50	50	0	100
University Exam Evaluation Scheme									
Internal Evaluation						External Evaluation			
Mid Sem (30 Marks)		20				Theory Exam		50	
Project of Software Testing		10							
Attendance		10				Practical Exam		NA	
Theory Assignment		10							
Total Marks		50				Total Marks		50	
Pre-requisites:									
Systems & Object-Oriented Design Methodologies									
Program Learning Outcome:									
Name of PO	Description								
PO1	Foundational & Theoretical Mastery								
PO2	Software Design & Development Expertise								
PO3	Problem-Solving & Analytical Ability								
PO4	Proficiency in Emerging Technologies								
PO5	Information Systems & Management Perspective								
PO6	Professional & Ethical Responsibility								
PO7	Communication & Teamwork								
PO8	Lifelong Learning & Career Readiness								
Course Learning Outcome:									



Name of CO	Description
CO1	Understand software engineering fundamentals, SDLC, process models, and the role of a software engineer.
CO2	Apply Agile methodology, Scrum, and Agile principles for iterative software development.
CO3	Analyze requirements, create SRS, and design software using UML diagrams and architecture principles.
CO4	Evaluate software quality, apply SQA standards, and perform software maintenance.
CO5	Apply software testing techniques and automation tools like Selenium to ensure reliable software.

Mapping of CO and PO:

Course Learning Objectives	Program Outcomes (POs)							
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8
CO1	3	2	1	1	1	2	1	1
CO2	2	2	3	2	2	1	1	2
CO3	2	3	3	2	2	1	1	2
CO4	2	2	2	3	3	2	1	2
CO5	1	2	3	2	3	1	1	2

Content:

Unit	Content	Weightage (%)	No of Hrs.	CO
Unit I	Foundations of Software Engineering: Introduction to Software, Software Engineering Overview, Role of Software Engineer, Characteristics of Good Software, Software Crisis, Software Development Life Cycle (SDLC), Software Process, Software Process Models (Waterfall, Incremental, Prototyping, Spiral, Agile Model) Introduction to Agile Methodology: Agility and Agile Process model, Extreme Programming, Agile Manifesto and Values, Agile Principles: Overview of the 12 principles of Agile software development, Scrum and Self-Organizing Teams.	20%	10	CO1
Unit II	Requirement Engineering & Design Concept: Understanding the Requirements, Requirement Modeling, Requirement Specification, and Requirements Engineering: Seven distinct tasks of requirement engineering, Gathering functional/non-functional requirements, Create SRS.	20%	10	CO2



	Software Design and Architecture: Design concepts and design principle, Architectural design, design UML diagrams: (Use case, class, sequence, activity diagram)			
Unit III	Software Quality Assurance and Software Maintenance: Quality Concept and Software Quality Assurance, Software Reviews (Formal, Technical Review), Software Reliability. The Quality Standards: ISO 9000, CMM, Six Sigma for SE, SQA plan Software Maintenance: Types of Software Maintenance, Re-Engineering, Reverse Engineering, Forward Engineering	20%	10	CO3
Unit IV	Software Testing Fundamentals: What is software testing, Why it is important? Verification, Validation, Qualification, STLC, Testing Basics: Levels: Unit, Integration, System, Acceptance, Types: Manual, Automation, functional, non-functional, Techniques: Black-box, White-box, Grey-box, Test Case Design: Techniques: Boundary Value Analysis, Equivalence Partitioning, Writing effective test plans and test cases	20%	10	CO4
Unit V	Advanced Testing and Automation Testing Tool (Selenium) Introduction to automation frameworks, How automation frameworks work? Types of automation frameworks, Where are automation frameworks used, Introduction to selenium, Features of selenium, Selenium limitations, Installation and Setup, Selenium, WebDriver Commands, Browser Commands, Navigation Commands, WebElement Commands, FindElement and FindElements Command CheckBox & Radio Button Operations, DropDown & Multiple Select Operations, Handle Dynamic WebTables in Selenium Webdriver, Locating web elements by Id, ClassName, LinkText, PartialLinkText, Name, TagName, CssSelector, XPath, Inspecting elements in web browsers.	20%	8	CO5



Reference Books:

1	Software Engineering: Roger S. Pressman & Bruce R. Maxim – <i>Software Engineering: A Practitioner's Approach</i>
2	Software Engineering: Ian Sommerville – <i>Software Engineering</i>
3	UML & Design: Craig Larman – <i>Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development</i>
4	UML & Design: Martin Fowler – <i>UML Distilled: A Brief Guide to the Standard Object Modeling Language</i>
5	Software Testing: Glenford J. Myers, Corey Sandler & Tom Badgett – <i>The Art of Software Testing</i>

Web Reference:

1	Software Engineering Institute (SEI), Carnegie Mellon University: https://www.sei.cmu.edu
2	Agile Alliance (Agile & Scrum concepts): https://www.agilealliance.org
3	Selenium Official Documentation (Automation Testing): https://www.selenium.dev/documentation

Question Paper Scheme:

	University Examination Duration: 2 Hours 30 minutes. SECTION-I Q-1: 10 Marks Q-2: 10 Marks Q-3: 10 Marks Q.4: 10 Marks Q.5: 10 Marks
--	--



L.J. INSTITUTES OF COMPUTER APPLICATIONS									
Programme		Master of Computer Applications				Branch/Spec.		Computer Applications	
Semester		II				Effective from Academic Year		2025-26	
Subject Category		Minor Elective - II				Effective for the batch Admitted in		June 2025	
Subject Code		240111206		Subject Name		Advanced Web Development using Laravel (Laravel)			
Teaching scheme						Examination scheme (Marks)			
(Lecture hours Per week)	Credit	L	T	P	Total	E	CEC	V	Total
4	4	0	1	6	48	0	50	50	100
University Exam Evaluation Scheme									
Internal Evaluation						External Evaluation			
Mid Sem Practical Exam (50 Marks)		30				Theory Exam		NA	
Project (50 Marks)		10							
Attendance		10				Practical Exam		50	
Total Marks		50				Total Marks		50	
Pre-requisites:									
Strong background and Knowledge of HTML, CSS, JavaScript and PHP is mandatory.									
Program Learning Outcome:									
Name of PO		Description							
PO1		Foundational & Theoretical Mastery							
PO2		Software Design & Development Expertise							
PO3		Problem-Solving & Analytical Ability							
PO4		Proficiency in Emerging Technologies							
PO5		Information Systems & Management Perspective							
PO6		Professional & Ethical Responsibility							
PO7		Communication & Teamwork							
PO8		Lifelong Learning & Career Readiness							



Course Learning Outcome:

Name of CO	Description
CO1	Understand the fundamentals of Laravel framework including MVC architecture, application structure, installation, configuration, and Artisan command-line tools.
CO2	Understand the concepts of Laravel routing, controllers, views, and Blade templating for developing dynamic web interfaces.
CO3	Understand database management in Laravel using migrations, Eloquent ORM, relationships, file upload, session handling, and CRUD operations.
CO4	Understand user authentication, authorization, and server-side validation techniques using built-in and custom validation rules for secure application development.

Mapping of CO and PO:

Course Learning Objectives	Program Outcomes (POs)							
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8
CO1	3	2	1	1	2	-	-	-
CO2	2	3	3	2	2	-	-	-
CO3	2	3	3	2	3	-	-	-
CO4	2	3	2	3	2	1	1	2

Content:

Unit	Content	Weightage (%)	No of Hrs.	CO
Unit I	Introduction to LARAVEL What is laravel, features of laravel MVC architecture, structure of laravel application (laravel directory structure). Installation Basic requirements for laravel, using laravel installer, using composer, how does composer work? Installation on Windows, finding and installing new packages Configuration Introduction, environment configuration, protecting sensitive configuration, maintenance mode, database configuration (setting database connection parameter for laravel and artisan) Artisan Artisan command line tool, database creation, artisan Migration, migration structure, creation migration, database seeding ,custom artisan commands, calling artisan commands in Normal ode, database creation, artisan, Tinker	20%	10	CO1



Unit II	Laravel Routing, Controllers Route definitions, route groups, controllers, route model binding, route caching, form method spoofing, csrf protection, redirects, aborting the request, custom responses Laravel views Laravel views, passing data to view Blade template : Template inheritance, master layout, extending the master layout, display variables, blade conditional statements, blade loops, executing php functions in blade	20%	10	CO2
Unit III	Laravel Migration Laravel migration, migration structure, generating migrations, migration commands Laravel file upload, laravel session ,Laravel database Laravel database, laravel eloquent, laravel relationship model, laravel tinkler Eloquent orm eloquent orm models: Naming conventions, table name & primary keys, timestamps Basic operations: create, retrieve, update, delete using models, displaying data from models in views	20%	10	CO3
Unit IV	User Authentication and Authorization : Using the auth() global helper and the auth façade Validation Defining the routes, creating the controller, writing the validation logic, displaying the validation errors, array validations, creating new validators, error messages & custom errors Available validators: Accepted, after (date), alpha, alpha dash, alpha numeric, array, before (date), between, boolean, date, date format, different, digits, digits between, e-mail, exists(database), image (file), integer, max, min, not in, numeric, regular expression, required, string custom validation rules.	20%	8	CO4
Unit V	Web Application Development using Laravel Self-paced final project work	20%	10	



Reference Books:	
1	Matt Stauffer, “LARAVEL Up and Running, A framework for building modern PHP Apps”, 1st Edition, O'REILLY (ISBN: 978-1-491-93608-5)
2	Martin Bean, “Laravel 5 Essentials”, Packet Publishing, ISBN 978-1-78528-301-7
3	Andy Turner, “Learning Laravel: A Beginner’s Guide to Building Web Applications with Laravel”, Packt Publishing, ISBN: 978-1-78439819-3
4	Terry McNavage, “Laravel Application Development Cookbook”, Packt Publishing, ISBN: 978-1-78588-151-0
Web Reference:	
1	https://www.tutorialspoint.com/laravel/index.htm
2	https://www.javatpoint.com/laravel
3	https://laravel.com/docs/7.x/quickstart
Question Paper Scheme:	
	University Examination Duration: 2 Hours 30 minutes. SECTION-I Q-1: Website design (40 Marks) SECTION-II VIVA (10 Marks)



L.J. INSTITUTES OF COMPUTER APPLICATIONS									
Programme		Master of Computer Applications				Branch / Spec.		Computer Applications	
Semester		II				Effective from Academic Year		2025-26	
Subject Category		Minor Elective - II				Effective for the batch Admitted in		June 2025	
Subject Code		240111207		Subject Name		No-Code Web Development (NoCode)			
Teaching scheme						Examination scheme (Marks)			
(Lecture hours Per week)	Credit	L	T	P	Total	E	CEC	V	Total
4	4	0	1	6	48	0	50	50	100
University Exam Evaluation Scheme									
Internal Evaluation						External Evaluation			
Mid Sem Practical Exam (50 Marks)		30				Theory Exam		NA	
Project (50 Marks)		10							
Attendance		10				Practical Exam		50	
Total Marks		50				Total Marks		50	
Pre-requisites:									
Program Learning Outcome:									
Name of PO		Description							
PO1		Foundational & Theoretical Mastery							
PO2		Software Design & Development Expertise							
PO3		Problem-Solving & Analytical Ability							
PO4		Proficiency in Emerging Technologies							
PO5		Information Systems & Management Perspective							
PO6		Professional & Ethical Responsibility							
PO7		Communication & Teamwork							
PO8		Lifelong Learning & Career Readiness							
Course Learning Outcome:									



Name of CO	Description
CO1	Understand no-code web development concepts and ecosystem
CO2	Design responsive UI/UX layouts using no-code tools
CO3	Develop and manage websites using no-code builders
CO4	Build basic no-code web applications with data and automation
CO5	Deploy, manage and maintain no-code web solutions

Mapping of CO and PO:

Course Learning Objectives	Program Outcomes (POs)							
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8
CO1	3	2	2	3	2	1	1	2
CO2	2	3	2	2	1	1	2	2
CO3	2	3	2	3	2	1	2	2
CO4	2	3	3	3	2	1	2	3
CO5	2	3	2	2	3	2	2	3

Content:				
Unit	Content	Weightage (%)	No of Hrs.	CO
Unit I	Introduction to No-Code Web Development - Web development overview - Static vs dynamic websites - Website vs web application - No-code development concept - No-code vs low-code vs coding - Industry applications & job roles - Overview of no-code ecosystem Tools (Demo Only) - Tool landscape overview (Figma, Bubble, WordPress, Zapier) - Account creation on required tools	10%	5	CO1
Unit II	UI/UX Design Using Figma - UI/UX principles - Layout, spacing, typography - Responsive design basics - Wireframing for websites - Design-to-development workflow Practical (Figma) - Create website wireframe - Design homepage & inner pages	25%	11	CO2



	• Mobile and desktop layouts Tool Used: Figma (Free version)			
Unit III	Dynamic Website Development Using WordPress • WordPress fundamentals • CMS concept • Pages, posts, categories • Theme & plugin concept • SEO and forms • Dynamic content handling Practical • Install WordPress (local/cloud) • Design website using Elementor • Create blog & contact form • Apply basic SEO Tools Used: • WordPress (Free) • Elementor (Free)	25%	11	CO3
Unit IV	No-Code Web Application Development Using Bubble • Web application concept • Bubble interface • Database creation • User authentication • Workflow logic • CRUD operations Practical • Build a simple web app • User signup/login • Data create, read, update, delete Tool Used: Bubble (Free tier)	20%	11	CO4
Unit V	Backend Data & Automation Using Google Sheets and Zapier • Google Sheets as database • Connecting Sheets with Bubble / WordPress • Automation concepts • Trigger-action workflows Practical • Store form data in Sheets • Send automated emails • Sync data between apps Tools Used: • Google Sheets (Free) • Zapier (Free tier)	20%	10	CO5
Reference Books:				



1	<i>Low-Code/No-Code: Citizen Developers and the Surprising Future of Business Applications</i> — Phil Simon (Suggested Reading) Phil Simon
2	<i>Mastering No-Code: Create Professional Quality Apps Without Coding (Vol. 1)</i> — Paul E Love (Reference) Karpagam Academy of Higher Education
3	<i>Designing and Prototyping Interfaces with Figma</i> — Fabio Staiano (Reference for UI/UX) Karpagam Academy of Higher Education
Web Reference:	
1	Official documentation & tutorials of Figma, WordPress, Bubble, Zapier
2	Free curated GitHub no-code learning collections GitHub
Question Paper Scheme:	
	University Examination Duration: 2 Hours 30 minutes. SECTION-I Q-1: (10 Marks) Q-2: (15 Marks) Q-3: (15 Marks) SECTION-II VIVA (10 Marks)

L.J. INSTITUTES OF COMPUTER APPLICATIONS			
Programme	Master of Computer Applications	Branch/Spec.	Computer Applications
Semester	II	Effective from Academic Year	2025-26
Subject Category	Minor Elective - II	Effective for the batch Admitted in	June 2025



Subject Code		240111208		Subject Name		Internet of Things (IOT)			
Teaching scheme						Examination scheme (Marks)			
(Lecture hours Per week)	Credit	L	T	P	Total	E	CEC	V	Total
4	4	0	1	6	48	0	50	50	100
University Exam Evaluation Scheme									
Internal Evaluation						External Evaluation			
Mid Sem Practical Exam (50 Marks)		30				Theory Exam		NA	
Project (50 Marks)		10							
Attendance		10				Practical Exam		50	
Total Marks		50				Total Marks		50	
Pre-requisites:									
Basic concept of Computer Networks must be clear with layered architecture and protocols. Types of networks with their connectivity components and constraints.									
Program Learning Outcome:									
Name of PO		Description							
PO1		Foundational & Theoretical Mastery							
PO2		Software Design & Development Expertise							
PO3		Problem-Solving & Analytical Ability							
PO4		Proficiency in Emerging Technologies							
PO5		Information Systems & Management Perspective							
PO6		Professional & Ethical Responsibility							
PO7		Communication & Teamwork							
PO8		Lifelong Learning & Career Readiness							
Course Learning Outcome:									
Name of CO		Description							
CO1		To gain knowledge on the bases of IoT and get familiarise the concepts of IoT to apply it in different applications.							
CO2		To gain knowledge of IoT Architecture, and the Protocols related to IoT							
CO3		To understand the concept of the Web and Internet to use it with IoT							
CO4		To extend the understanding of web communication protocols used by IoT/ M2M devices. And illustrate the usage of messaging protocols between connected devices and the web							
CO5		To develop the software components of IoT system using Arduino/Raspberry							



Pi Programming.

Mapping of CO and PO:

Course Learning Objectives	Program Outcomes (POs)							
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8
CO1	1	2	2	2	3	3	2	2
CO2	1	2	2	1	2	2	3	2
CO3	1	2	3	1	3	2	3	3
CO4	1	2	3	1	3	2	2	3
CO5	1	2	1	1	2	2	2	1

Content:

Unit	Content	Weightage (%)	No of Hrs.	CO
Unit I	Definition of Internet of Things, Introduction to M2M, Application areas of IoT, Characteristics of IoT, Things in IoT, IoT challenges, IoT levels	20%	10	CO1
Unit II	Introduction to Arduino Platform Utilization of controller boards: Arduino (Nano/ Uno) - Pin Diagram and Schematic Diagram, NodeMCU (ESP8266)/ESP-32) - Pin Diagram and schematic diagram Introduction of embedded C, Python (micro python) programming	20%	10	CO2
Unit III	Introduction to Sensors, Microcontrollers and Interfacing Introduction to Sensor: Sensor Interfacing, Types of Sensors, Controlling Sensors through Webpages - A brief on hand exposure of study sensors from application point of view: Expected parameter for on hand usage of sensors - IR, PIR, Ultrasonic, Sound, Fire, Gas, Temperature, Humidity, Wind speed etc... Basic operations: create, retrieve, update, delete using models, displaying data from models in views	20%	10	CO3
Unit IV	Introduction of different Communication Protocols and Platforms Local Area Network, Ethernet Shield, Wi-Fi, HTTP, MQTT (Message Queue Telemetry Transport), Introduction of cloud platform, Thingspeak, AWS.	20%	10	CO4



Unit V	Application development using Arduino	20%	8	CO5
	Self-paced Final Project Work			

Reference Books:	
1	Internet of Things by Shriram Vasudevan, Nagrajan and Sundaram, Wiley India
2	Internet of Things with ARDUINO and BOLT, Ashwin Pajankar, BPB
3	Rajkamal,” Internet of Things”, Tata McGraw Hill publication
4	21 IoT Experiments, Yashavant Kanetkar, Shrirang Korde, BPB
5	Vijay Madiseti and Arshdeep Bahga, “Internet of things(A-Hand-on-Approach)” 1st Edition, Universal Press
6	INTERNET OF THINGS WITH RASPBERRY PI AND ARDUINO By Rajesh Singh, Anita Gehlot, Lovi Raj Gupta, Bhupendra Singh, and Mahendra Swain – CRC Press
Web Reference:	
1	https://www.tutorialspoint.com/internet_of_things/index.htm
2	https://www.iotworldtoday.com/
3	https://aws.amazon.com/iot/
4	https://www.cisco.com/c/en_in/solutions/internet-of-things/overview.html
5	https://www.cisco.com/c/en_in/solutions/internet-of-things/iot-network-connectivity.html
Question Paper Scheme:	
	University Examination Duration: 2 Hours 30 minutes. SECTION-I Create Project using IOT Sensor Q-1: Circuit connection(10 Marks) Q-2: Coding (15 Marks) Q-3: Working output demonstration (15 Marks) SECTION-II VIVA (10 Marks)



L.J. INSTITUTES OF COMPUTER APPLICATIONS									
Programme		Master of Computer Applications				Branch/Spec.		Computer Applications	
Semester		II				Effective from Academic Year		2025-26	
Subject Category		AEC(Ability Enhancement Course)				Effective for the batch Admitted in		June 2025	
Subject Code		240114209		Subject Name		Communication and Technical Report Writing (CTRW)			
Teaching scheme						Examination scheme (Marks)			
(Lecture hours Per week)	Credit	L	T	P	Total	E	CEC	V	Total
2	3	2	1	0	24	50	50	0	100
University Exam Evaluation Scheme									
Internal Evaluation						External Evaluation			
Mid Sem Theory Exam (30 Marks)		30				Theory Exam		50	
Assignment/Presentation		10							
Attendance		10				Practical Exam		NA	
Total Marks		50				Total Marks		50	
Pre-requisites:									
Program Learning Outcome:									
Name of PO		Description							
PO1		Foundational & Theoretical Mastery							
PO2		Software Design & Development Expertise							
PO3		Problem-Solving & Analytical Ability							
PO4		Proficiency in Emerging Technologies							
PO5		Information Systems & Management Perspective							
PO6		Professional & Ethical Responsibility							
PO7		Communication & Teamwork							
PO8		Lifelong Learning & Career Readiness							
Course Learning Outcome:									
Name of CO		Description							
CO1		Understand the basics of communication, its types, barriers, and communication							



	networks for effective interpersonal and professional interaction.
CO2	Prepare professional written documents such as business letters, resumes, cover letters, and LinkedIn profiles using standard formats.
CO3	Draft professional emails using AI tools ethically and effectively.
CO4	Apply appropriate professional etiquettes in telephonic communication, cross-cultural situations, workplace behavior, and time management.
CO5	Demonstrate effective oral communication skills through active listening, interviews, structured presentations, and audience handling.

Mapping of CO and PO:

Course Learning Objectives	Program Outcomes (POs)							
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8
CO1	3	-	2	-	2	1	3	2
CO2	2	-	2	1	2	2	3	3
CO3	1	-	2	3	1	3	2	3
CO4	1	-	2	-	2	3	3	2
CO5	1	-	2	-	1	2	3	2

Content:

Unit	Content	Weightage (%)	No of Hrs.	CO
Unit I	FUNDAMENTALS OF PROFESSIONAL COMPETENCY 1. Communication as a Core Professional Competency • Concept and Definition of Professional Communication • Role of Communication in IT Industry and Corporate Environment. • Importance of non verbal communication in corporate. 2. Audience-Driven Communication • Meaning and Importance of Audience Analysis • Types of Audiences - o Technical vs Non-Technical Audience - o Internal vs External Audience - o Individual vs Group Audience 3. Barriers to Professional Communication • Physical and Organizational Barriers • Psychological and Cultural Barriers • Language and Semantic Barriers • Strategies to Overcome Communication Barriers	20%	4	CO1



Unit II	FUNCTIONAL AND WORKPLACE WRITING SKILLS. 1Introduction to Functional Writing • Meaning and Importance of Functional Writing 2. Functional Writing Types • Notices and Circulars • Reports (Short and Informal) • Requests, Complaints, and Apologies 3. Scenario-Based Functional Writing • Writing for Workplace Scenarios - o Requesting Leave or Resources - o Reporting Technical Issues - o Responding to Client Queries - o Writing Incident and Status Reports • Tone, Format, and Clarity in Functional Writing	20%	6	CO2
Unit III	AI- ASSISTED PROFESSIONAL WRITING SKILLS 1. Introduction to AI Tools for Writing • Meaning of Artificial Intelligence (AI) in simple terms • Use of AI tools in daily and professional writing • Examples of AI writing tools (e.g., ChatGPT) • Benefits of using AI for students and professionals 2. Professional Email Writing with AI Support - o Job Application Email - o Internship / Training Request Email - o Leave and Permission Email - o Follow-up Email 3. Using AI to Draft Emails • Giving simple instructions (prompts) to AI • Generating email drafts using AI • Editing AI-generated emails 4. Resume Writing with AI Assistance • Purpose of a Resume • Introduction to Activity-Based Resume • Formatting and basic resume design	20%	6	CO3
Unit IV	ETIQUETTES - o Telephone etiquettes - o Etiquettes for foreign business trip - o Visits of foreign counterparts - o Etiquettes for small talks	20%	5	CO4



	○ Respecting privacy ○ Learning how to say NO ○ Time management			
Unit V	ORAL COMMUNICATION AND PRESENTATION Active listening question handling Presentation Planning Outlining Delivery Techniques handling Q&A	20%	3	CO5